TEACHING GUIDE

ไกด์การจัดการเรียนการสอน

30204-2003 โครงสร้างข้อมูลและอัลกอริทึม · ปวส.1/1

แผนกเทคโนโลยีสารสนเทศ | วิชาทฤษฎี+ปฏิบัติ | ปวส.1 | ภาคเรียน 1/2569 | อ้างอิงผู้เรียน 36/46 คน

1 ภาพรวมผู้เรียนและทิศทางการสอน

- ผู้เรียนห้องนี้เรียนรู้ได้ดีผ่านการลงมือทำ (Kinesthetic) (35%) รองลงมาคือการเห็นภาพ (Visual) (29%)
- ความสนใจเด่นของห้องคือนักคิดวิเคราะห์ (Investigative) และนักริเริ่ม (Enterprising) → เหมาะกับการเรียนรู้โดยใช้ปัญหาเป็นฐาน (Problem-based / Inquiry)
- มีผู้เรียน Fixed Mindset ถึง 50% → ต้องให้ความสำคัญกับการเสริมแรงเชิงบวกและงานที่สำเร็จได้เร็ว
- วิชาทฤษฎี+ปฏิบัติ กับผู้เรียนสายลงมือทำ → ใช้รูปแบบ “อธิบายสั้น 10 นาที → ลงมือทำทันที” สลับทั้งคาบ

AI ไกด์เชิงลึกเฉพาะรายวิชา (Gemini)

AI วิเคราะห์จากสถิติรวมของห้องและเนื้อหาวิชา (ไม่ส่งชื่อนักเรียน) · เรียก AI เฉพาะเมื่อกดปุ่ม และจำผลไว้ใช้ซ้ำ

ผู้เรียนห้องนี้มีความโดดเด่นด้านการลงมือทำ (K 35%) และการเห็นภาพ (V 29%) สอดคล้องกับความถนัดเชิงวิเคราะห์ (I 31%) จึงเหมาะกับการจัดการเรียนรู้แบบเห็นการทำงานจริงเชื่อมโยงกับการเขียนโค้ด อย่างไรก็ตามมีสัดส่วน Fixed Mindset สูงถึง 50% ซึ่งอาจทำให้ผู้เรียนถอดใจได้ง่ายเมื่อเจอความซับซ้อนของอัลกอริทึมหรือข้อผิดพลาดในการรันโค้ด ทิศทางการสอนจึงต้องเน้นการแปลงนามธรรมให้เห็นภาพจริง ค่อยๆ ลดการช่วยเหลือผ่านโจทย์บันไดเวียน และสร้างพื้นที่ที่มองข้อผิดพลาดเป็นโอกาสในการเรียนรู้

กลยุทธ์การสอน

Dual Coding ผสานโมเดลจำลองกับการเขียนโค้ด

สอนมโนทัศน์ของโครงสร้างข้อมูลด้วยการจำลองกายภาพและภาพกราฟิกก่อนแปลงเป็นโค้ด Python เช่น การใช้กระดาษโพสต์อิทจำลองการ Push/Pop ของ Stack หรือการชี้ตำแหน่งโหนดใน Linked List ด้วยปากกาก่อนเขียนคลาสโหนด เพื่อช่วยกลุ่มชอบการมองเห็นและการลงมือทำจับต้องนามธรรมได้ชัดเจน

เหมาะกับ: ทั้งห้อง โดยเฉพาะกลุ่มเน้นการเห็นภาพ (V) และลงมือทำ (K)

Worked Examples และ Faded Guidance

ลดภาระการรับรู้ (Cognitive Load) ในหัวข้อที่ซับซ้อน เช่น การเขียนฟังก์ชัน Recursion สำหรับ Binary Tree โดยให้โค้ดตัวอย่างที่สมบูรณ์พร้อมอธิบาย Trace Table ก่อน จากนั้นให้ผู้เรียนเติมเต็มบางฟังก์ชัน (Faded Example) และขยับไปสู่การเขียนเองจากศูนย์

เหมาะกับ: ทั้งห้อง โดยเฉพาะผู้เรียนกลุ่ม Fixed Mindset ที่กลัวความล้มเหลว

Retrieval Practice ด้วยการไล่การทำงานแบบ Dry-Run

จัดกิจกรรมสั้นๆ ต้นคาบ 5-10 นาที ให้ผู้เรียนเขียนผลลัพธ์ของการท่องไปในกราฟ (BFS/DFS) หรือการสลับที่ใน Bubble Sort ด้วยกระดาษและดินสอโดยไม่ใช้คอมพิวเตอร์ เพื่อฟื้นคืนความจำและตรวจสอบความเข้าใจระดับตรรกะก่อนเขียนโปรแกรม

เหมาะกับ: ทั้งห้อง โดยเฉพาะกลุ่มเน้นการคิดวิเคราะห์ (I)

Peer Debugging & Algorithmic Challenge

แบ่งกลุ่มขนาดเล็กให้ผู้เรียนร่วมกันค้นหาจุดผิดพลาด (Bug) และเปรียบเทียบ Time Complexity ของการค้นหาแบบ Linear Search และ Binary Search ผ่านการวัดเวลาทำงานจริงในโปรแกรม ช่วยกระตุ้นทักษะการทำงานร่วมกันและเปิดโอกาสให้แลกเปลี่ยนแนวคิด

เหมาะกับ: ผู้เรียนสายสืบเสาะวิเคราะห์ (I) และสายชอบการลงมือปฏิบัติ (K)

สื่อ/เครื่องมือที่แนะนำ

- **VisuAlgo (visualgo.net)** — ใช้แสดงการทำงานแบบแอนิเมชันของ Sorting, Binary Search Tree และ Graph Algorithms เพื่อให้ผู้เรียนเห็นภาพกลไกและตัวชี้ตำแหน่งอย่างเป็นขั้นตอน
- **Python Tutor (pythontutor.com)** — ใช้ Trace การทำงานของตัวแปร Pointer และ Frame สแตกในฟังก์ชัน Recursion รวมถึงการชี้ Memory Address ของ Linked List ในภาษา Python
- **Google Colab** — ใช้เป็นเครื่องมือฝึกเขียนโค้ดและทดสอบวัดเวลาประมวลผล (time execution) เพื่อเปรียบเทียบ Big-O Notation โดยไม่ต้องติดตั้งโปรแกรมลงเครื่อง
- **Kahoot! หรือ Quizizz** — ใช้สำหรับกิจกรรม Low-stakes Quiz เพื่อทบทวนนิยามและลำดับขั้นตอนของอัลกอริทึม (Retrieval Practice) ก่อนเริ่มการเขียนโค้ดภาคปฏิบัติ

การวัดผลที่เหมาะสม

- การประเมินแบบย่อยระหว่างเรียน (Formative Assessment) ผ่านการเขียน Trace Table และการแก้โจทย์เติมโค้ดสั้นๆ รายสัปดาห์
- แล็บทดสอบรายบุคคล (Hands-on Practical Exam) เน้นการแก้ไขบั๊กและการปรับแต่งโครงสร้างข้อมูลให้ทำงานได้ถูกต้องตามเงื่อนไขที่กำหนด
- มินิโปรเจกต์คู่ (Mini-Project) ให้เลือกโครงสร้างข้อมูลที่เหมาะสมไปประยุกต์แก้ปัญหาจริง เช่น ระบบคิวในร้านค้า หรือระบบประวัติการทำงานด้วย Stack พร้อมนำเสนอผลการวิเคราะห์ประสิทธิภาพ

ความเข้าใจผิดที่พบบ่อยและวิธีแก้

อาร์เรย์และลิงก์ลิสต์ · เข้าใจว่าการเข้าถึงข้อมูลตัวที่ n ของ Linked List ทำได้เร็วเท่ากับ Array Indexing (O(1))

แก้โดย: ให้ผู้เรียนทดลองเขียนลูปนับก้าวเพื่อเดินตามพอยน์เตอร์จากโหนดเริ่มต้นทีละโหนด เปรียบเทียบกับการดึงข้อมูลผ่านดัชนีของอาร์เรย์ เพื่อให้เห็นข้อจำกัด O(n) เชิงประจักษ์

การเรียงลำดับและการค้นหา · เข้าใจว่า Binary Search ใช้ได้กับชุดข้อมูลทุกรูปแบบโดยไม่คำนึงว่าข้อมูลเรียงลำดับแล้วหรือไม่

แก้โดย: ให้ชุดข้อมูลตัวเลขที่กระจายตัวแบบไม่เรียงลำดับ แล้วนำอัลกอริทึม Binary Search ไปค้นหาข้อมูลที่ตกหล่น เพื่อให้ผู้เรียนวิเคราะห์สาเหตุด้วยตนเองว่าทำไมจึงต้อง Sort ก่อน Search

ความซับซ้อนของอัลกอริทึม (Big O) · สับสนระหว่างจำนวนบรรทัดของโค้ดกับเวลาในการประมวลผล คิดว่าโค้ดสั้นกว่าต้องเร็วกว่าเสมอ

แก้โดย: เปรียบเทียบโค้ดสั้นที่มีลูปซ้อนลูปแบบ O(n^2) กับโค้ดที่ยาวกว่าแต่เป็น O(n) โดยป้อนข้อมูลขนาดล้านตัวแล้วจับเวลาจริงใน Python

ผู้เรียน Fixed Mindset: รับมือกับกลุ่ม Fixed Mindset (50%) ด้วยการเปลี่ยนกรอบความคิดต่อข้อผิดพลาด (Syntax Error/Logic Error) ให้เป็นส่วนหนึ่งของการเรียนรู้ โดยให้คะแนนตามกระบวนการคิดและการดีบัก (Process-oriented Grading) มากกว่าความถูกต้องในรอบแรก ใช้โจทย์แบบแบ่งขั้นบันไดจากง่ายไปยากเพื่อสร้าง Small Wins สะสมความมั่นใจ และหลีกเลี่ยงการชมว่า 'ฉลาด' แต่เปลี่ยนเป็นชื่นชมความพยายามในการไล่หาสาเหตุของข้อผิดพลาด

สร้างโดย gemini-3.8-flash · 27 ก.ย. 2569 19:20 · ใช้ Key ของคุณ · 2434 tokens · เนื้อหาจาก AI ควรตรวจทานก่อนนำไปใช้

2 สื่อ เครื่องมือ และกิจกรรมที่แนะนำ

เรียงตามสัดส่วนความถนัดของห้อง · ใช้ทุกแบบในวิชา แต่ให้น้ำหนักกับแบบ “หลัก” และ “รอง” มากที่สุด

การลงมือทำ (Kinesthetic)

หลัก · 35%

สื่อ/เครื่องมือ

- แล็บเขียนโปรแกรมย่อย (Mini Lab) พร้อมโค้ดตั้งต้น
- กิจกรรมไม่ใช้คอมพิวเตอร์ (CS Unplugged) เช่น การ์ดบัตรตัวเลข

กิจกรรม

- Code-along: พิมพ์ตามครูแล้วทดลองแก้ค่า
- โจทย์ Parsons: เรียงบรรทัดโค้ดที่สลับกันให้ทำงานถูก
- มินิโปรเจกต์ที่เห็นผลทันที

การเห็นภาพ (Visual)

รอง · 29%

สื่อ/เครื่องมือ

- เว็บจำลองการทำงานของอัลกอริทึม เช่น VisuAlgo
- Python Tutor แสดงค่าตัวแปร/หน่วยความจำทีละบรรทัด
- Flowchart และแผนภาพโครงสร้างข้อมูลบนกระดาน

กิจกรรม

- วาดแผนภาพสถานะข้อมูลก่อน-หลังแต่ละคำสั่ง
- ใช้สีแยกประเภทข้อมูล/พอยน์เตอร์ในสไลด์และโค้ด

การอ่าน-เขียน (Read/Write)

เสริม · 18%

สื่อ/เครื่องมือ

- ใบความรู้/สรุปเนื้อหาเป็นข้อ ๆ
- Pseudocode และเอกสารอ้างอิงภาษา Python

กิจกรรม

- เขียน Pseudocode ก่อนเขียนโค้ดจริง
- ใบงานอ่านโค้ดแล้วเขียนอธิบาย/ทำนายผลลัพธ์

การฟัง-พูดคุย (Aural)

เสริม · 18%

สื่อ/เครื่องมือ

- คลิปอธิบายสั้น/พอดแคสต์ของครู
- การอธิบายโค้ดแบบคิดดัง ๆ (Think-aloud)

กิจกรรม

- จับคู่เขียนโปรแกรม (Pair Programming) ผลัดกันอธิบายให้คู่ฟัง
- ให้ผู้เรียนอธิบายการทำงานของโค้ดด้วยคำพูด (Code Walkthrough)

3 วิธีสอนที่เหมาะกับความสนใจของห้อง

ผู้เรียนสายนักคิดวิเคราะห์ (Investigative) · 31%

การเรียนรู้โดยใช้ปัญหาเป็นฐาน (Problem-based / Inquiry)

เริ่มด้วยโจทย์ปัญหา ให้ตั้งสมมติฐาน ทดลอง วิเคราะห์ว่าทำไมผลออกมาแบบนั้น

ผู้เรียนสายนักริเริ่ม (Enterprising) · 17%

การแข่งขันและการนำเสนอ (Challenge / Pitching)

โจทย์ท้าทายแข่งเป็นทีม มีบทบาทผู้นำ นำเสนอผลงานต่อผู้อื่น

▶ **บทบาทในงานกลุ่มตามความสนใจ (ใช้ตอนแบ่งหน้าที่)**

4 แผนกิจกรรมรายหน่วยการเรียน

กิจกรรมสำหรับ 2 สไตล์หลักของห้อง ในแต่ละหัวข้อ · แก้ไขหัวข้อได้ที่ "แก้ไขบริบทรายวิชา"

หน่วย	หัวข้อ	การลงมือทำ (Kinesthetic)	การเห็นภาพ (Visual)
1	**อาร์เรย์**	นักเรียนยืนเรียงแถวเป็นช่องอาร์เรย์ ถือป้ายตัวเลข ทดลองแทรก–ลบจริง	วาดช่องอาร์เรย์พร้อมเลขดัชนี แสดงการเข้าถึง/เลื่อนข้อมูลตอนแทรก–ลบ

หน่วย	หัวข้อ	การลงมือทำ (Kinesthetic)	การเห็นภาพ (Visual)

แผนการสอนหน่วย “อาร์เรย์” · 150 นาที

จุดประสงค์: ผู้เรียนสามารถอธิบายหลักการจัดสรรหน่วยความจำแบบเรียงติดกันของอาร์เรย์ และเขียนโปรแกรมภาษา Python เพื่อเข้าถึง แทรก และค้นหาข้อมูล พร้อมระบุความซับซ้อนของเวลา (Time Complexity) เบื้องต้นได้อย่างถูกต้อง

15 นาที **ขั้นนำเข้าสู่บทเรียนและทบทวนความรู้เดิม (Retrieval Practice** ภาพ ฟัง
ครูยกตัวอย่างเปรียบเทียบ 'ตู้ล็อกเกอร์ที่มีหมายเลขกำกับ' กับตัวแปรเดี่ยว จากนั้นให้นักเรียนตอบคำถามสั้น เพื่อทบทวนชนิดข้อมูล List ใน Python แล้วสาธิตภาพเปรียบเทียบการเก็บข้อมูลแบบกระจัดกระจายกับการจัดเก็บแบบเรียงลำดับต่อเนื่องในหน่วยความจำ

30 นาที **ขั้นสร้างมโนทัศน์และสาธิตผ่านตัวอย่าง (Dual Coding & Worked** ภาพ อ่านเขียน
ครูอธิบายโครงสร้าง 1D และ 2D Array การคำนวณ Index (เริ่มต้นที่ 0) และสาธิตการทำงานผ่าน PythonTutor.com ให้นักเรียนเห็นการเลื่อนตำแหน่ง (Shift) ของข้อมูลเมื่อทำการแทรกหรือลบ พร้อมวิเคราะห์ว่าเหตุใดการเข้าถึงข้อมูลจึงเป็น O(1) แต่การแทรกเป็น O(n)

35 นาที **ขั้นฝึกปฏิบัติแบบมีโครงเสริมการเรียนรู้ (Faded Worked Exampl** อ่านเขียน ลงมือทำ
ผู้เรียนเปิด Google Colab หรือ Thonny ทำใบงานเขียนโค้ดแบบเติมคำในช่องว่าง (Faded Examples) เช่น การเติมเงื่อนไขลูปตรวจสอบ Index Out of Range และการเขียนฟังก์ชันค้นหาข้อมูลแบบ Linear Search โดยครูเดินให้คำแนะนำกระตุ้นการคิดทีละขั้น

45 นาที **ขั้นแก้โจทย์ประยุกต์และลงมือปฏิบัติจริง (Active Learning Cha** ภาพ ลงมือทำ
ให้นักเรียนจับคู่จำลองบทบาท 'โปรแกรมเมอร์' (เน้นกลุ่มชอบวิเคราะห์ I และลงมือทำ K) แก้โจทย์ระบบบันทึกคะแนนสอบ: ให้เขียนโค้ดเพิ่ม-ลบคะแนน และค้นหาคะแนนสูงสุด/ต่ำสุด พร้อมบันทึกผลการทดสอบกรณีใส่ข้อมูลผิดพลาดเพื่อฝึกดีบักโค้ดด้วยตนเอง

25 นาที **ขั้นสรุปผลและสะท้อนการเรียนรู้ (Reflection & Assessment)** ฟัง ลงมือทำ
นักเรียนร่วมกันทำควิซสั้น 4 ข้อผ่าน Quizizz เพื่อวัดความเข้าใจ ครูเน้นย้ำข้อผิดพลาดที่พบบ่อย (Common Bugs) และเปิดโอกาสให้นักเรียนแชร์วิธีการแก้ปัญหาที่ค้นพบในคาบ

ปรับตามสไตล์ผู้เรียน

ภาพ: ใช้แผนภาพกล่องหน่วยความจำที่มีหมายเลข Index กำกับชัดเจน และใช้ PythonTutor แสดงภาพจำลองการเคลื่อนย้ายตำแหน่งข้อมูลแบบแอนิเมชัน

ฟัง: ครูใช้วิธีพูดบรรยายกระบวนการคิดทีละขั้น (Think-aloud) และจัดกิจกรรมให้ผู้เรียนจับคู่อธิบายตรรกะของโค้ดให้เพื่อนฟังก่อนเริ่มลงมือพิมพ์

อ่านเขียน: แจกเอกสารสรุปไวยากรณ์ (Cheat sheet) รวบรวมคำสั่งการเข้าถึง Index, ฟังก์ชันพื้นฐานของ List/Array และตารางสรุป Time Complexity ให้อ่านประกอบ

ลงมือทำ: ให้พิมพ์โค้ดทดสอบจริงทีละบรรทัดใน Interactive Shell เพื่อสังเกต Error ทันที และมีกิจกรรมสลับบทบาทสลับกันแก้โจทย์หน้าคีย์บอร์ด

สื่อ/อุปกรณ์

- Google Colab หรือ Thonny IDE
- เว็บไซต์ PythonTutor.com (สำหรับแสดงการทำงานของหน่วยความจำ)
- สไลด์แผนภาพหน่วยความจำ (Visual Memory Map)
- ระบบควิซออนไลน์ Quizizz หรือกระดาษคำถามสั้น
- ใบงานปฏิบัติการโค้ดแบบเติมช่องว่าง (Faded Scratchpad)

คำถามทบทวนท้ายคาบ

1. หากอาร์เรย์มีขนาด 8 ช่อง ข้อมูลตัวแรกและตัวสุดท้ายจะอยู่ที่ Index ใด และหากพยายามเข้าถึง Index ที่ 8 จะเกิดอะไรขึ้น?
2. เพราะเหตุใดการเข้าถึงข้อมูลในอาร์เรย์ด้วย Index จึงใช้เวลาคงที่ O(1) ในขณะที่การแทรกข้อมูลตัวแรกใช้เวลา O(n)?
3. การจัดเก็บข้อมูลของอาร์เรย์ใน RAM มีลักษณะทางกายภาพแตกต่างจากการเก็บข้อมูลตัวแปรเดี่ยวกระจัดกระจายอย่างไร?
4. ในการค้นหาข้อมูลในอาร์เรย์ขนาด N ช่องที่ยังไม่เรียงลำดับ ในกรณีแย่ที่สุด (Worst-case) โปรแกรมต้องเปรียบเทียบข้อมูลกี่รอบ?

ผู้เรียน Fixed Mindset: สร้างวัฒนธรรม 'Error คือเรื่องปกติของการเป็นโปรแกรมเมอร์' โดยครูยกตัวอย่างข้อผิดพลาด IndexError: list index out of range ที่ทุกคนต้องเจอและวิธีแก้ พร้อมใช้ใบงานแบบค่อยๆ ปลดล็อกความยาก (Scaffolding) ชื่นชมที่ความพยายามในการไล่โค้ด (Tracing process) มากกว่าการได้ผลลัพธ์ถูกในรอบแรก

หน่วย	หัวข้อ	การลงมือทำ (Kinesthetic)	การเห็นภาพ (Visual)

สร้างโดย gemini-3.8-flash · 27 ก.ย. 2569 19:22 · ควรตรวจทานก่อนนำไปใช้

2	**ลิงก์ลิสต์**	นักเรียนจับมือเป็นโหนด ใช้เชือกแทนพอยน์เตอร์ แล้วสลับ/ตัดต่อ	แผนภาพโหนด-ลูกศร เปลี่ยนสีพอยน์เตอร์ทุกขั้นของการแทรก/ลบ
3	**สแตก**	ใช้แก้วซ้อน/กล่องจริงทำ push-pop · แล็บตรวจวงเล็บด้วย stack	ภาพกองจาน/แอนิเมชัน push-pop บน VisuAlgo
4	**คิว**	จำลองต่อแถวซื้อของในห้อง แล้วเขียนโปรแกรมจำลองคิว	แผนภาพคิวแบบวงกลม (circular queue) พร้อมตำแหน่ง front/rear
5	**ทรี**	บัตรตัวเลขให้นักเรียนสร้าง BST บนพื้น/กระดาน แล้วเดินท่องทรีจริง	วาด Binary Search Tree ทีละโหนดด้วยสี และแสดงเส้นทางการท่อง (traversal)
6	**กราฟ**	ใช้หมุด+เชือกสร้างกราฟ แล้วเล่นเกมหาเส้นทางสั้นสุด	แผนที่เส้นทางในวิทยาลัยเป็นกราฟ ระบายสีโหนดที่เยี่ยมแล้วตอน BFS/DFS
7	**การค้นหา**	หาไพ่ในสำรับที่เรียงแล้ว/ยังไม่เรียงแข่งกัน	แอนิเมชันเทียบ linear search กับ binary search จำนวนขั้นที่ใช้
8	**การเรียงลำดับ**	นักเรียนถือป้ายตัวเลขแล้วเรียงตัวเองตามอัลกอริทึม · จับเวลาเทียบกัน	VisuAlgo เทียบ bubble / selection / insertion / merge sort

✦ ทุกหน่วย: ควิซสั้นต้นคาบถัดไปเพื่อทบทวน (Retrieval Practice)

5 การสร้างบรรยากาศและกรอบความคิด (Mindset)

สำหรับทั้งห้อง

- ชมที่ความพยายามและกลยุทธ์ ไม่ใช่ความเก่ง (“เธอลองหลายวิธีจนได้”)
- ทำให้ความผิดพลาดเป็นเรื่องปกติ เช่น แชร์บั๊กที่น่าสนใจประจำสัปดาห์
- ให้แก้งานได้หลังรับฟีดแบ็ก (resubmission)

ผู้เรียน Fixed Mindset (50%)

- เริ่มด้วยงานง่ายที่สำเร็จได้เร็ว (Quick Wins) แล้วค่อยเพิ่มความยาก
- แบ่งงานใหญ่เป็นเป้าหมายย่อยที่เห็นความก้าวหน้า
- ให้ฟีดแบ็กแบบ “ยังไม่ได้ … แต่ใกล้แล้ว” แทนผ่าน/ไม่ผ่าน
- จับคู่กับเพื่อน Growth Mindset ในงานกลุ่ม

6 กลุ่มผู้เรียนและแนวทางซ่อมเสริมรายกลุ่ม

คลิกกลุ่มเพื่อดูรายชื่อ · ใช้ประกอบการซ่อมเสริมและดูแลรายบุคคล

กลุ่มควรดูแลเป็นพิเศษ: Fixed Mindset · 18 คน

เริ่มด้วยงานง่ายที่สำเร็จได้เร็ว (Quick Wins) แล้วค่อยเพิ่มความยาก
แบ่งงานใหญ่เป็นเป้าหมายย่อยที่เห็นความก้าวหน้า
ให้ฟีดแบ็กแบบ “ยังไม่ได้ … แต่ใกล้แล้ว” แทนผ่าน/ไม่ผ่าน
จับคู่กับเพื่อน Growth Mindset ในงานกลุ่ม

6801011 อนุชา ศรีสุข · 6801012 อรอุมา ใจงาม · 6801014 กัญญารัตน์ ทองดี · 6801015 จิรายุ แก้วมณี
6801018 ดวงใจ พรหมมา · 6801020 นันทิชา ศรีสะเกษ · 6801021 ปฐมพงศ์ คำภา · 6801022 ปาริฉัตร ภูมิใจ
6801023 พงศกร ศรีสุข · 6801026 มณีรัตน์ ทองดี · 6801027 รัฐภูมิ แก้วมณี · 6801028 รุ่งนภา บุญมา
6801030 วิมลรัตน์ พรหมมา · 6801031 ศุภชัย จันทร์หอม · 6801033 สิทธิชัย คำภา · 6801034 สุนิสา ภูมิใจ
6801039 ธนวัฒน์ แก้วมณี · 6801044 ปิยะนุช ศรีสะเกษ

กลุ่มการลงมือทำ (Kinesthetic) · 13 คน

ซ่อมเสริมด้วยแล็บย่อย/มินิโปรเจกต์ ไม่ใช่การให้อ่านหนังสือเพิ่ม

6801013 กฤษดา มั่นคง · 6801023 พงศกร ศรีสุข · 6801024 พรทิพย์ ใจงาม · 6801026 มณีรัตน์ ทองดี
6801027 รัฐภูมิ แก้วมณี · 6801030 วิมลรัตน์ พรหมมา · 6801038 ไอลดา ทองดี · 6801039 ธนวัฒน์ แก้วมณี
6801041 ชยพล สายทอง · 6801042 กนกวรรณ พรหมมา · 6801043 วีรภัทร จันทร์หอม · 6801044 ปิยะนุช ศรีสะเกษ
6801045 สรวิชญ์ คำภา

กลุ่มการเห็นภาพ (Visual) · 9 คน

ซ่อมเสริมด้วยวิดีโอ/แผนภาพสรุปทีละขั้น และภาพจำลองการทำงาน

6801012 อรอุมา ใจงาม · 6801014 กัญญารัตน์ ทองดี · 6801015 จิรายุ แก้วมณี · 6801018 ดวงใจ พรหมมา
6801021 ปฐมพงศ์ คำภา · 6801029 วรเมธ สายทอง · 6801034 สุนิสา ภูมิใจ · 6801040 เบญจวรรณ บุญมา
6801046 ธิดารัตน์ ภูมิใจ

กลุ่มการฟัง-พูดคุย (Aural) · 4 คน

ซ่อมเสริมด้วยการติวแบบพูดคุย เพื่อนสอนเพื่อน ให้อธิบายด้วยคำพูดตัวเอง

6801017 ณัฐพล สายทอง · 6801020 นันทิชา ศรีสะเกษ · 6801031 ศุภชัย จันทร์หอม · 6801033 สิทธิชัย คำภา

กลุ่มการอ่าน-เขียน (Read/Write) · 5 คน
ซ่อมเสริมด้วยใบสรุปและแบบฝึกหัดเขียนที่มีเฉลยอธิบาย

6801019 ธีรเดช จันทร์หอม 6801032 ศิริพร ศรีสะเกษ 6801035 อภิสิทธิ์ ศรีสุข 6801036 อัญชลี ใจงาม
6801037 เอกชัย มั่นคง

กลุ่มเรียนรู้หลายช่องทาง (Multimodal) · 5 คน
ใช้สื่อผสมหลายรูปแบบ ปรับตัวได้ดีกับทุกกิจกรรม

6801011 อนุชา ศรีสุข 6801016 ชลธิชา บุญมา 6801022 ปาริฉัตร ภูมิใจ 6801025 ภานุวัฒน์ มั่นคง
6801028 รุ่งนภา บุญมา

ยังไม่มีข้อมูล (ยังไม่เล่นเกม) · 10 คน
นัดให้เล่นเควสต์ในคาบถัดไป (ใช้เวลาประมาณ 10–15 นาทีต่อเควสต์) เพื่อให้ไกด์ครอบคลุมทุกคน

6901001 กิตติ นักเรียน 6901002 ขวัญใจ นักเรียน 6901003 คมสัน นักเรียน 6901004 จันทร์เพ็ญ นักเรียน
6901005 ชัยวัฒน์ นักเรียน 6901006 ณัฐธิดา นักเรียน 6901007 ธนพล นักเรียน 6901008 นภัสสร นักเรียน
6901009 ปิยะพงษ์ นักเรียน 6901010 พิมพ์ชนก นักเรียน

7 หลักการสอนที่ใช้ได้กับผู้เรียนทุกคน

มีหลักฐานงานวิจัยรองรับ — ใช้เป็นฐานร่วมกับข้อ 2–4

ตัวอย่างที่ทำให้ดูทีละขั้น (Worked Examples)

สาธิตวิธีคิดทีละขั้นให้ดูครบ 1–2 ตัวอย่าง แล้วค่อย ๆ ลดขั้นที่ช่วยจนผู้เรียนทำเองได้ (faded examples)

เหตุผล: ลดภาระความจำของผู้เริ่มต้น ได้ผลดีมากกับเนื้อหาที่เป็นขั้นตอน

ทบทวนด้วยการดึงความจำ (Retrieval Practice)

ควิซสั้น 3–5 ข้อต้นคาบจากเนื้อหาครั้งก่อน ไม่นับคะแนน

เหตุผล: การนึกออกเองช่วยให้จำได้นานกว่าการอ่านซ้ำ

ภาพคู่คำอธิบาย (Dual Coding)

ทุกแนวคิดสำคัญมีทั้งแผนภาพและคำอธิบายควบคู่กัน

เหตุผล: ช่วยทุกคน ไม่เฉพาะผู้เรียนสายภาพ

การเรียนรู้เชิงรุก (Active Learning)

บรรยายไม่เกิน 15 นาทีต่อช่วง แล้วสลับเป็นกิจกรรม/คำถามให้คิด-จับคู่-แลกเปลี่ยน (Think-Pair-Share)

เหตุผล: งานวิจัยชี้ว่าลดอัตราสอบตกและเพิ่มการมีส่วนร่วม

ฝึกแบบเว้นระยะ (Spaced Practice)

แบ่งแบบฝึกหัดเรื่องเดิมกระจายหลายคาบ แทนการฝึกรวดเดียว

เหตุผล: ความรู้คงทนกว่าการอัดในครั้งเดียว

8 การติดตามผล (สำหรับ ว.PA / วิจัยชั้นเรียน)

- ทดสอบก่อน-หลังเรียนรายหน่วย แล้วเปรียบเทียบคะแนนเฉลี่ยและร้อยละผู้ผ่านเกณฑ์
- บันทึกการมีส่วนร่วม/การส่งงานแยกตามกลุ่มผู้เรียน (VARK/Mindset) ก่อนและหลังปรับการสอน
- ให้ผู้เรียนเล่นเควสต์อีกครั้งภาคเรียนถัดไป เพื่อดูการเปลี่ยนแปลงของ Mindset
- แบบสอบถามความพึงพอใจต่อรูปแบบการสอนที่ปรับ

หมายเหตุ: สไตล์การเรียนรู้ (VARK) ใช้เป็นข้อมูลประกอบการเลือกสื่อเพื่อเพิ่มความสนใจและการมีส่วนร่วม งานวิจัยปัจจุบันไม่ยืนยันว่าการสอนตรงสไตล์เพียงแบบเดียวทำให้ผลสัมฤทธิ์สูงขึ้น จึงแนะนำให้สอนหลายรูปแบบ โดยเน้นแบบที่ห้องถนัด ร่วมกับหลักการสอนที่มีหลักฐานรองรับ